


THE UNIVERSITY of ADELAIDE

# Pattern Mining and Recommender System

## Group: 39

Shashikant Chaudhary:

Joel Varghese Thomas:

Anoop Ajith:

**The University of Adelaide**

Mining Big Data - COMP SCI 7306

Lecturer: Dr. Alfred Krzywicki

April 27, 2025

# Contents


**1 Executive Summary** **2**

**2 Introduction** **2**
2.1 Topic . . . 2
2.2 Problem . . . 2
2.3 Purpose . . . 2
2.4 Report Structure . . . 3

**3 Exploratory Data Analysis** **3**
3.1 Dataset Overview . . . 3
3.2 Analysis of User Behavior . . . 3
3.3 Popular Products . . . 4
3.4 Time Related Analysis . . . 4
3.5 Basket Analysis . . . 5

**4 Implementation** **6**
4.1 Task 1: Frequent Pattern Mining . . . 6
4.1.1 Method Selection and Implementation . . . 6
4.1.2 Determining the Thresholds . . . 7
4.1.3 Scoring Patterns . . . 8
4.2 Task 2: Collaborative Filtering . . . 9
4.2.1 Recommendation from Raw Data . . . 9
4.2.2 Recommendation Method from Frequent Patterns . . . 10
4.2.3 Evaluation Metrics . . . 10
4.3 Task 3: System Integration . . . 11
4.3.1 Literature Review on Pattern-Based Recommendations . . . 11
4.3.2 Integration Approach . . . 12

**5 Testing and Results** **13**

**6 Conclusions and Recommendations** **14**
6.1 Key Findings . . . 14
6.2 Method Recommendation . . . 15
6.3 Limitations . . . 15
6.4 Significance . . . 15

**7 Reflections** **16**
7.1 (Shashi) . . . 16
7.2 (Joel Varghese Thomas) . . . 17
7.3 (Anoop Ajith) . . . 17

# 1 Executive Summary

This report presents a comprehensive analysis and functioning of a recommender system for a grocery retail store. The system consists of two major components: pattern mining and the other is collaborative filtering. By using these two modules together, a personalized recommendation of items is provided to the user. The overall aim of this system is to boost the sales of grocery retail stores by providing relevant recommendations to the user based on their previous purchasing pattern and popular items.

The approach used by us combines two approaches: pattern mining and collaborative filtering. The pattern mining module discovers the popular and frequently bought items together; on the other hand, collaborative filtering provides recommendations based on similar purchasing patterns between users. A key feature introduced in our system is 'recency', this concept helped us to generate a recommendation based on the recent purchasing history.

Our finding shows that the integration of pattern mining with a collaborative filter actually fetches better results than a collaborative filter alone. This is corroborated with the results in the testing section. The difference is negligible but could be improved using a better integration approach.

In the grocery retail business, our prototype has significant business implications. By generating relevant suggestions for the users, retailers can improve their sales and customer satisfaction significantly. The simple UI of this system is easy to understand and generates relevant recommendations that can be integrated with the existing retailer's customer management system for personalized user recommendations.

# 2 Introduction

## 2.1 Topic

Retail grocery stores generate vast amounts of data through everyday sales, this data is saved into their system by the POS(Point of Sale) system. This data is a very useful resource and can be mined to understand the purchasing behavior of the customers. Large retail market businesses want to leverage this data to improve their sale and customer experience and gain more percentage in market share.

## 2.2 Problem

Assessment scenario omitted from this public copy.

## 2.3 Purpose

The main purpose is to develop a prototype of a recommendation system that uses data from both frequent patterns and collaborative filtering and generates personalized recommendations for a user. The main goal is to check if the recommendation engine can provide better recommendations using both frequent pattern mining and collaborative filtering, rather than just a standalone collaborative filter. The goals of the project are as follows:

- Mining and identifying the frequent items from the data

- Providing recommendations alone with a collaborative filter.
- Providing recommendations using both frequent patterns and collaborative filters.
- Evaluate the results of both approaches.
- Integrate and create a UI for the recommendation engine.

### 2.4 Report Structure

This report is structured as follows:

- **Exploratory Data Analysis**: Examines the transaction dataset to understand shopping patterns, item relationships, and temporal trends
- **Implementation**: Details the development of frequent pattern mining, collaborative filtering, and system integration components
- **Testing and Results**: Presents performance metrics and comparative analysis of the recommendation approaches
- **Conclusions and Recommendations**: Summarizes key findings and provides recommendations for implementation

## 3 Exploratory Data Analysis

Data understanding is a very crucial step before creating any kind of system, in this section we have shown various data analyses to understand patterns present in the data, which helped as a guide for making our recommender system.

### 3.1 Dataset Overview

The dataset contains transactions of 1 year from jan 2014 to jan 2015 with over 19,382 valid transactions. These records have the following attributes:

- User_id: Unique customer ID from the loyalty program
- Date: transaction date
- itemDescription: product purchased
- Temporal attributes: Date, year, month, day, day of week

Table 1: Dataset Statistics

| Metric | Value |
|---|---|
| Total valid transactions | 19,382 |
| Unique users | 3,493 |
| Unique products | 167 |
| Date range | Jan 2014 - Jan 2015 |

### 3.2 Analysis of User Behavior

From this analysis summarised in Figure 1, we have found that the majority of the users have very few transactions in the given training dataset, most of the users just have single or just 2-3 transactions. This makes it very challenging for the system to provide recommendations due to limited purchase history, and it needs to leverage the pattern mining approach.

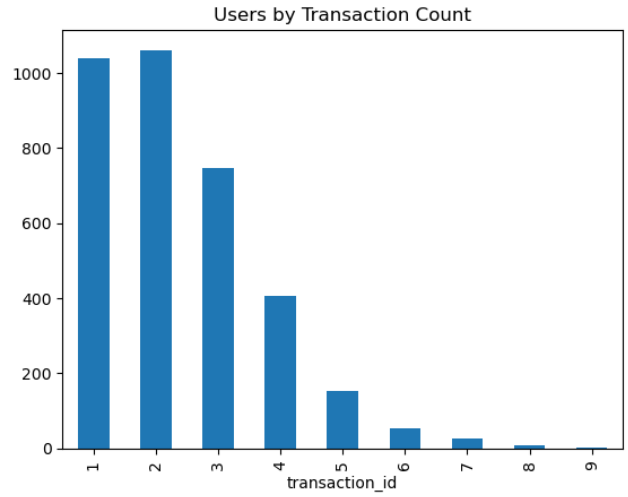



Figure 1: Users by transaction count distribution

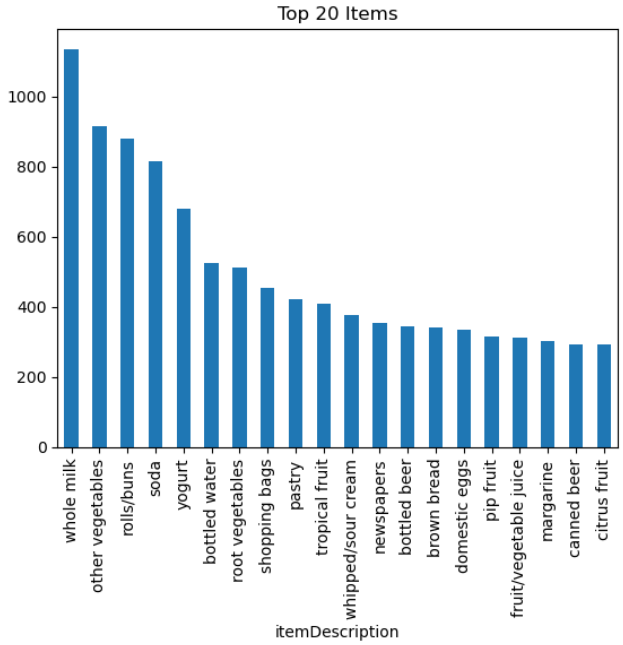



Figure 2: Top 20 most frequently purchased items

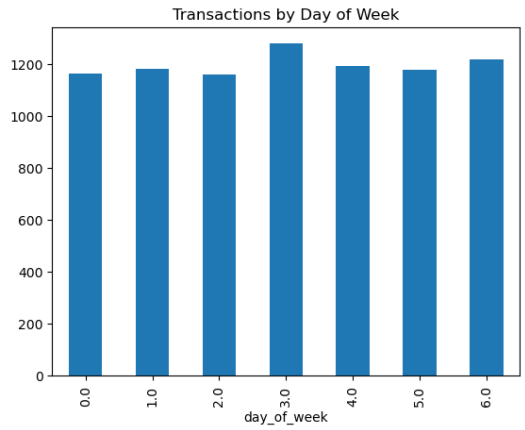
## 3.3 Popular Products

Upon analyzing the distribution of the top 20 items presented in the Figure 2, we have found that whole milk is the most frequent item bought, and then followed by other significant frequent items: "other vegetables," "rolls/buns," and "soda. It is clear that the distribution is not uniform, and a few items are found frequently in the majority of transactions.

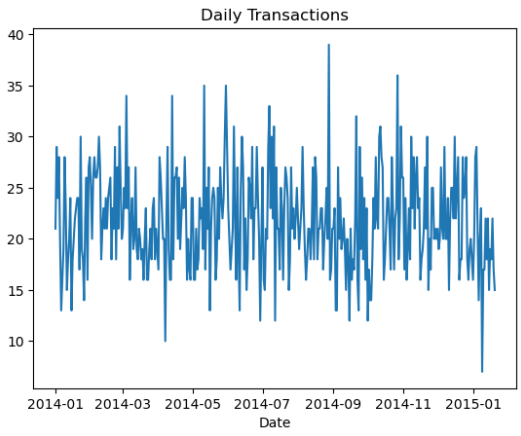
The dominance of these few popular items makes the recommendation both easy and difficult, because these items provide a strong signal to our pattern mining, but just recommending based on these frequent items may not be very useful.

## 3.4 Time Related Analysis

We have performed various time related analysis on our transaction data, which provides important insights for the recommendation approaches.




(a) Transactions by day of week




(b) Daily transactions over time

Figure 3: Temporal transaction patterns

Figure 3a informs us that transaction or shopping activity is consistent each day; it just shows a slight increase in the transaction on Wednesday, and there is no specific day trend to the data. The Figure 3b shows that the daily transaction numbers fluctuate between 15-35 transactions per day, and this fluctuation of the pattern does not show any trend or seasonality in the shopping patterns over the year.

We have done an even more detailed analysis by plotting a heat map of transaction count by day of week and month 4, and we have found some interesting patterns, which show higher purchases in the month of January, probably due to post-holiday shopping behaviour or new year sales.

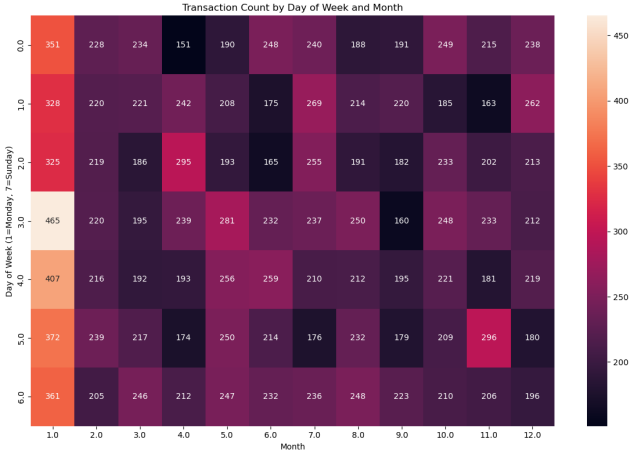



Figure 4: Transaction count by day of week and month

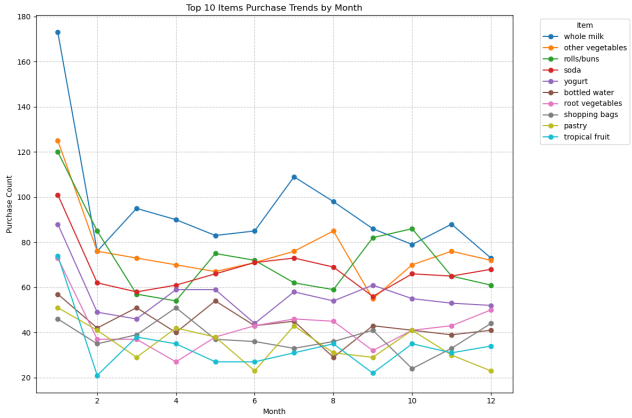



Figure 5: Purchase trends for top items by month

We have also plotted the trend of the top 10 popular items as shown in Figure 5 and found some seasonality with products like "whole-milk" showing the rise in its sales during January and July. It can also be observed that sales of every product drop in February; these types of temporal patterns in the data suggest the importance of recency integration in our recommendation system.

## 3.5 Basket Analysis

On items per transaction(Basket Size), we have found that most of the transactions contain 2 items(approx 75%), and a significant portion contains 3 items(approx 17%) as shown in the Table 2.

Table 2: Distribution of basket sizes

| Items per transaction | Number of transactions | Percentage |
|---|---|---|
| 1 | 131 | 1.57% |
| 2 | 6,297 | 75.31% |
| 3 | 1,443 | 17.26% |
| 4 | 446 | 5.33% |
| 5+ | 44 | 0.53% |

This basket-size distribution suggests focusing on mining patterns containing 2 or 3 items, as it represents the majority of the transactions.

Further insights can be taken from Figure 6, which is a heatmap showing items frequently bought together.

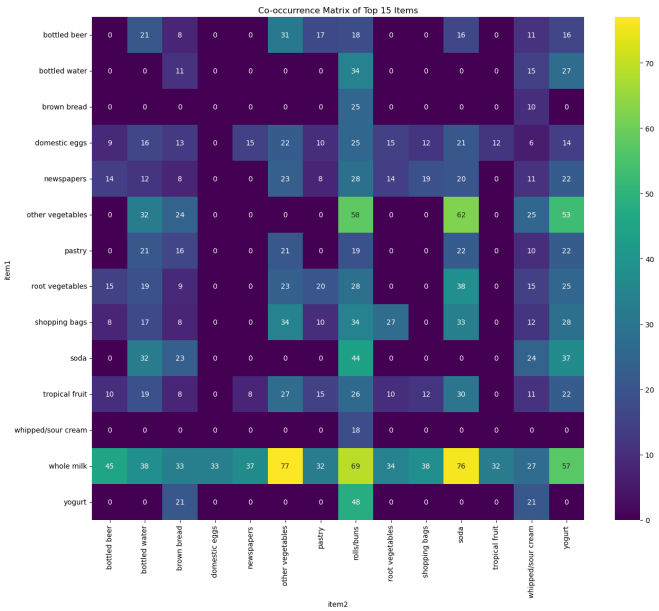



Figure 6: Co-occurrence matrix of top 15 items

Some key analysis from the co-occurrence heatmap are as follows:

- We can see that whole "milk" co-occurs very frequently with "other vegetables"(77 occurrences).
- "other vegetables" also co-occur with "rolls/buns"(58) and "soda"(62).
- Some items like "whipped/sour" cream rarely co-occur with other products
- Moreover, this is a very sparse matrix and shows that most of the items rarely co-occur or do not co-occur at all.
- This matrix helps in understanding the association rules from our pattern mining results.

This analysis on co occurrence of items will help in guiding and validating our pattern mining approach.

# 4 Implementation

## 4.1 Task 1: Frequent Pattern Mining

### 4.1.1 Method Selection and Implementation

For the frequent itemset mining component, we have chosen the FP-Growth algorithm over Apriori because it is more efficient and faster for big datasets, does not generate all possible candidate items rather uses an FP-tree to find frequent items[5].

The main components of the FP-Growth algorithm are as follows:

1. Preparing the transactions and performing one-hot encoding using mlxtend's Transaction Encoder.
2. Finding out the frequent itemsets using the FP-Growth algorithm.
3. Generating association rules using the discovered frequent itemsets.

We have used mlxtend's library for implementing the FP-Growth algorithm and using these parameters:

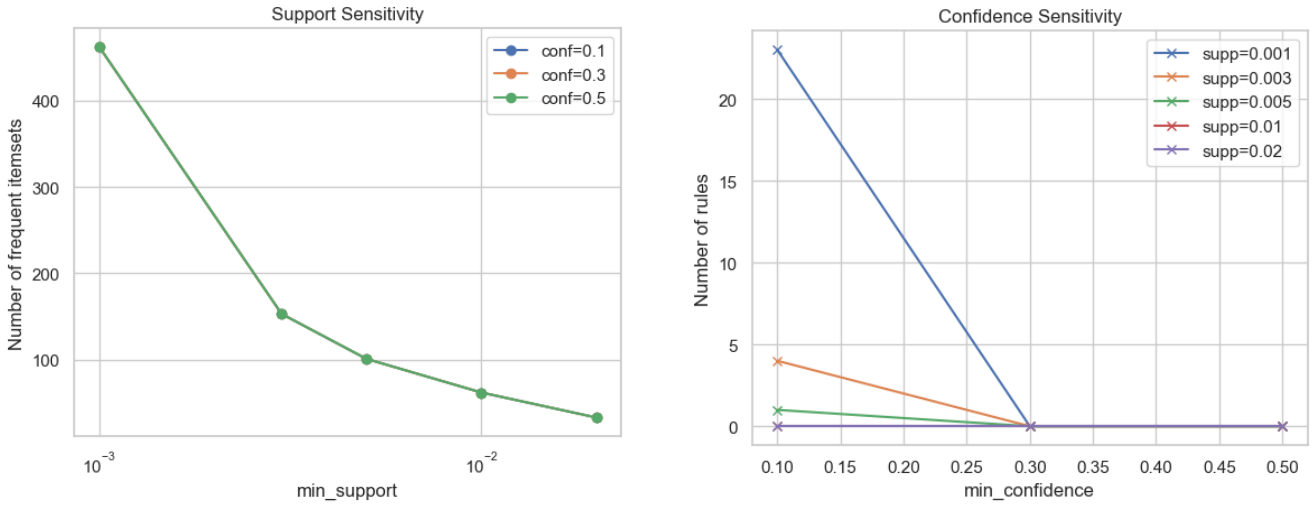
- `min_support`: Minimum support value for generating the frequent items.
- `max_len`: Length of frequent items occurring together, by default we have kept it at 5 so that longer rules could be generated.

After finding frequent itemsets, it was then passed into the association rule generation function to generate the rules based on a configurable minimum confidence metric.

### 4.1.2 Determining the Thresholds

In order to find out the optimal values of support, we have plotted the support sensitivity graph(Figure 7a), which shows a number of patterns generated at different support values at varying min support values.

Later, we also plotted a confidence sensitivity graph(Figure 7b), which shows a number of association rules generated at different confidence values and fixed support.




(a) Support threshold effect on number of itemsets (b) Confidence threshold effect on number of rules

Figure 7: Parameter sensitivity analysis

As shown in Figure 7, we have observed the following :

- The number of frequent itemsets decreases significantly on increasing the support level.
- Around 0.001 support, approximately 400 itemsets are found.
- Around 0.02 support, only 40(approx) itemsets are founds.
- Most of the support values generated very few rules.
- Only at support 0.001, 20+ rules are generated.
- The rules drop to zero when the confidence level is above 0.3.

Upon analysing these results, we have found the optimal support value for support and confidence ranging between 0.001-0.005 and 0.07-0.1, respectively. These values help us to get a sufficient number of patterns while maintaining the overall quality.

In the Figure 8, we have analyzed how more recent data affect the pattern discovery.

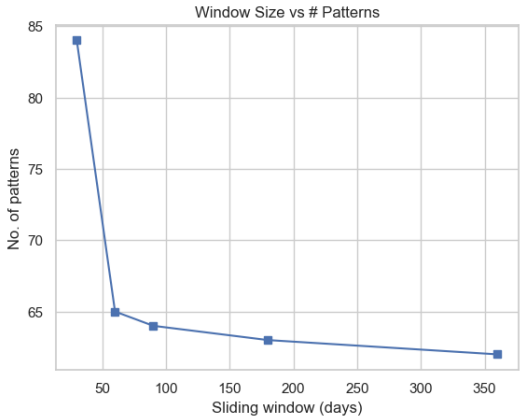



Figure 8: Effect of time window size on number of patterns discovered

From this graph, we have come to the conclusion that shorter recent data between 30-90 days fetches maximum patterns. This analysis encouraged us to integrate recency weighting in the pattern scoring[4].

### 4.1.3 Scoring Patterns

We have introduced a different scoring mechanism that uses recency in weighting the patterns.

For every item $i$ purchased by the user, we have calculated the recent weight as follows.

$$w_i = e^{-0.005 \cdot (D_{\max} - D_i)} \tag{1}$$

where:

- $D_i$ = most recent purchase date of item $i$.
- $D_{\max}$ = most recent purchase date in the dataset.
- $\Delta_i = D_{\max} - D_i$ = no. of days since item $i$ was last purchased.
- $\lambda = 0.005$ = decay rate.

In case of an itemset $I$ containing more than 1 item, we calculate its recency weights as the mean of its constituent items.

$$\text{recencyScore}(I) = \frac{1}{|I|} \sum_{i \in I} w_{\text{recency}}(i) \tag{2}$$

Finally, we use a combination of support and recency with an adjustable percentage of them in the final scoring.

$$\text{score}(I) = \alpha \cdot \text{normSupport}(I) + (\beta) \cdot \text{recencyScore}(I) \tag{3}$$

where:

- $\alpha$ and $\beta$ represent the percentage of involvement in the final score.
- $\alpha + \beta = 1$; $\alpha$ and $\beta \in [0, 1]$
- normSupport$(I)$ represents the normalized support, which is scaled to $[0, 1]$.

The novel scoring approach that we have used helps us in addressing the pattern frequency and the customer's purchasing behaviour over time.

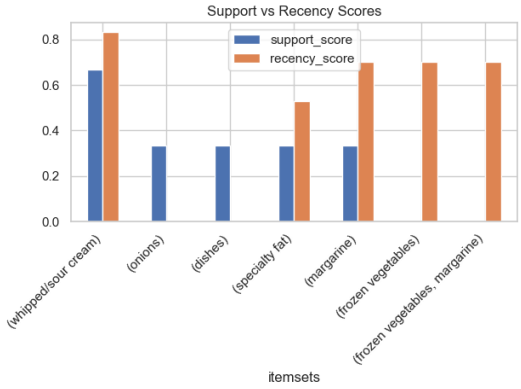



Figure 9: support-based and recency-enhanced pattern scoring

In Figure 9, we can see the comparison of support scoring and combined score(recency+support) on different itemsets based on the user 3681's purchasing history. It is clear that this method can significantly alter rankings. For example, the item "specially fat" has a higher recency score compared to just historical frequency, and it will be ranked differently based on that score.

## 4.2 Task 2: Collaborative Filtering

### 4.2.1 Recommendation from Raw Data

In the collaborative filtering component, we analyzed both Item-based and User-based collaborative filtering methods. The item-based method was chosen over the user-based method because of the greater stability of item relationships compared to user preferences over time, indicated by the RMSE score in the analysis.

The collaborative filtering algorithm consists of the following:

1. Creating the Recency weighted user-item matrix
2. Computing the item-item similarity matrix using cosine similarity and adjusted cosine similarity(adjusted cosine similarity was discarded due to poor results)
3. Computing and sorting the final score, generated by scoring based on the weighted user-item matrix and the user's interaction with similar items.
4. Address the cold-start problem for a new user by recommending the most popular items

The final recommendation score is based on the following formula:

$$\text{score}(u, i) = \sum_{j \in P_u} \text{sim}(i, j) \cdot \text{freq}(u, j) \tag{4}$$

where:

- u is the user
- j is the item
- $P_u$ is the set of items purchased by user $u$
- $\text{sim}(i, j)$ is the cosine similarity between items $i$ and $j$
- $\text{freq}(u, j)$ is the purchase frequency of item $j$ by user $u$

The cosine similarity function from the sklearn library is used to calculate the similarity matrix. To account for recent purchase history, we add a recency weight to the user-item matrix. For each user-item record, the recency weight is calculated as follows:

$$\Delta_i = t_{\max} - t_i \quad \text{(in days)} \qquad w_i = \exp\big(-\lambda\, \Delta_i\big)$$

where:

- $\Delta_i$ is the difference between the highest date and the date of transaction
- $w_i$ is the exponential decay since last purchase and $\lambda$ is the decay parameter

To address the cold-start problem where a new user does not have user-specific recommendations, we will be recommending the top 5 popular items based on the most bought items.

### 4.2.2 Recommendation Method from Frequent Patterns

The integration of frequent patterns with the collaborative filtering task resulted in two methods to combine recommendations. The first method works as follows:

1. Get the recommendations and respective normalized scores from CF task
2. Get the scored patterns and the normalized score from associative pattern mining task
3. Create a set with a combination of the items from 1. and 2.
4. For each item, set a **new score**(hybrid score) as the maximum score from tasks 1. and 2.
5. Sort the scores and return the top 5 recommendations. The new score generated is used for thresholding and filtering the top 5 recommendations.

$$\text{score}_{\text{combined}}(u, i) = max(\text{score}_{CF}(u, i), \text{score}_{FP}(u, i)) \tag{5}$$

where:

- $\text{score}_{CF}(u, i)$ is the normalized score from collaborative filtering
- $\text{score}_{FP}(u, i)$ is the normalized score from pattern mining

The second method produced better results and works as follows:

1. Get the recommendations and respective normalized scores from CF task
2. Get the scored patterns and the normalized score from associative pattern mining task
3. Filter the top 50 patterns from 2. and create a list where one item is in the purchase history and the other is not
4. The output from 3. is combined with CF output based on set intersection and scoring of top items in the set

After much discussion, we would ideally use the rules generated from Pattern Mining to determine the best recommendations and combine it with CF based on weighted scoring.

### 4.2.3 Evaluation Metrics

To evaluate the quality of the recommendations, the following metrics are used:

- **Precision**: A measure of the recommended items that are actually relevant from a user's purchases. test set)

$$\text{Precision} = \frac{|\text{Recommended Items} \cap \text{True Items}|}{|\text{Recommended Items}|} \tag{6}$$

- **Recall**: In this scenario, recall is the proportion of True items that are recommended to the user.

$$\text{Recall} = \frac{|\text{Recommended Items} \cap \text{True Items}|}{|\text{True Items}|} \tag{7}$$

- **F1 Score**: The harmonic mean of precision and recall, a combined measure of the recommender performance.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{8}$$

- **MAP score**: The Mean Average Precision score uses the Hit rate to calculate the Average Precision which is the same as precision but with an additional ranking to the order of recommendations.

The figure below summarizes the performance of the CF recommendations. The graph highlights that the User-Based recommendation metrics are skewed, especially the MAE and RMSE values. These values indicate that the user-based raw scores are skewed. We can see from the graph that user-based model perform very well over item-based but the raw scores from User based contribute to the errors. We will extend this to the integration section.

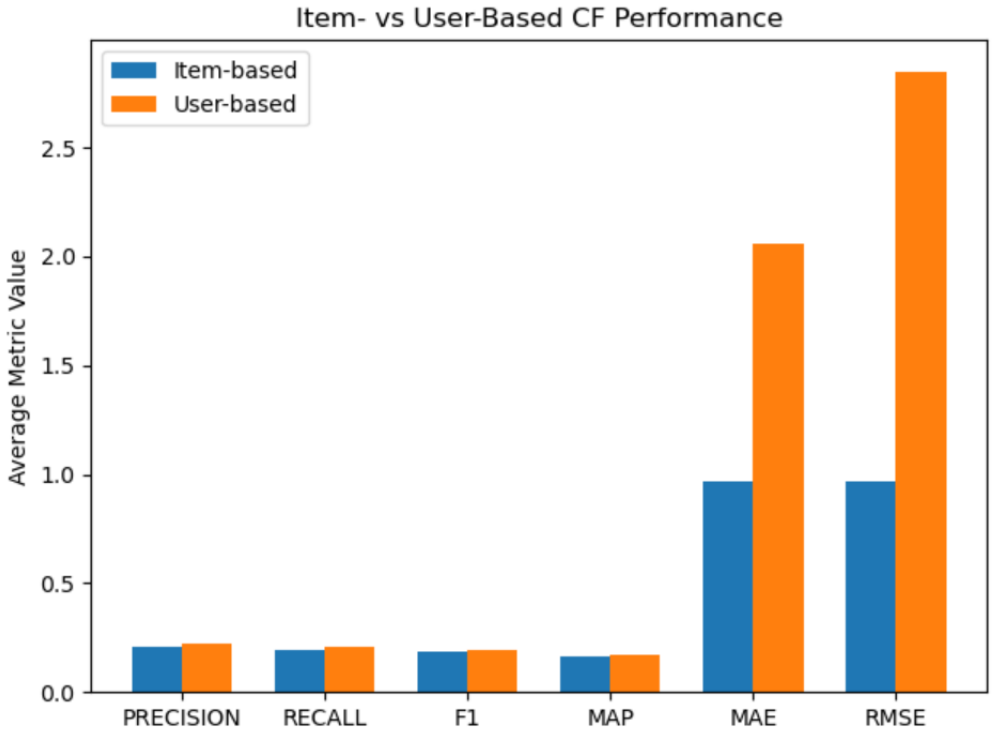



Figure 10: Item vs User based Collaborative Filtering Metrics

These metrics provide a comprehensive assessment of recommendation quality, capturing different aspects of performance.

## 4.3 Task 3: System Integration

### 4.3.1 Literature Review on Pattern-Based Recommendations

Retail analytics has embraced hybrid recommender systems that combine knowledge-driven pattern mining and data-driven collaborative filtering (CF) because they retain the personalisation that association-rule approaches alone cannot offer while mitigating the data-sparsity and cold-start issues of standalone CF. The study done on an online book recommendation system by using

collaborative filtering and association mining has shown gains in the accuracy metrics by using the hybrid approach, having sparse user data [7].

An early retail-focused hybrid was presented by Liu et al., who first group customers based on RFM (Recency, Frequency, Monetary) characteristics before mining *sequential association rules* within each group[1]. As tunable hyper-parameters, minimum support and confidence were handled as follows: increasing support from 0.3 % to 0.6 % reduced the rule base by 42 % with only a 3 % loss in precision, demonstrating the traditional coverage–noise trade-off. Stale purchases are automatically down-weighted due to RFM clustering's encoding of *recency*; the authors report a 4.8 % lift in $F_1$ over a time-agnostic CF baseline on six months' worth of supermarket receipts.

Choi et al. use temporal information in a different way. Their HOPE system combines recommendations from *sequential pattern analysis* (PrefixSpan) with item-based CF calculated from *implicit ratings*[2]. The patterns are mined over sliding windows of 30, 60, and 90 days, so window length is an additional sensitivity parameter even though they don't use an explicit decay function. A 60-day window with a 1 % support threshold maximised MAP in their click-and-buy data, proving that optimal parameters are domain-specific and gain from empirical tuning.

Viktoratos combines association rules and community knowledge to solve the user cold-start issue in context-aware recommenders[3]. Although their study lacks time awareness, it provides a thorough assessment of *rule-ranking metrics*. When comparing confidence, lift, conviction, and a probability-based composite, they demonstrate that ranking is just as important as thresholding: on a sparsified location-based-service dataset, a rule-quality score that maximised lift and confidence produced the highest top-5 precision (0.41). This realisation applies to our prototype, where hundreds of rules must be ranked before merging with CF scores after surviving a 0.5 % support filter.

Three recurring themes come out of these studies:

1. **Parameter sensitivity is inevitable.** None of the authors use "default" values; instead, they all experiment with support, confidence, and window size (where applicable).
2. "Recency is important, but it can be applied in different ways." Liu *et al.* incorporates recency into RFM clustering, Choi *et al.* depends on temporal windows, and Viktoratos *et al.* disregards time. Explicit time-decay or windowing is wise for grocery data, where product popularity varies seasonally, according to conflicting evidence.
3. **Single-method baselines are consistently surpassed by hybridisation.** Our choice to combine FP-Growth rules with item-based CF is supported by reported gains of 4–12% in $F_1$ or MAP.

### 4.3.2 Integration Approach

In this task, the pattern mining task and collaborative filtering task is combined using a top-level interface. The interface asks for the User ID input and whether to use frequent patterns or not. The interface calls the top-level recommendation function with the user id and pattern mining decision parameters. This top-level function covers multiple pieces:

1. The function addresses the cold start problem where a user does not exist in the training set. This is solved by using the singletons with the highest support score from task 1 or through the top most bought items from Task 2.
2. Now, if pattern mining is enabled using the "up" parameter:
   (a) The functions defined in Task 1 are called and the top 5 patterns are selected based on the pattern score.

(b) For each itemset that is in the top 5 patterns, we check whether the items are in the user purchase history. If not, we add it to the relevant recommendation.

(c) The Collaborative Filtering function is called to get the top 5 recommendations.

(d) The items from part b. and c. are combined using max scoring and the top 5 recommendations are filtered out.

3. If Pattern Mining is disabled, only the collaborative filtering function is called to get the top 5 recommendations.

The integration code also does the following tasks:

- The Testing and evaluation of the methodology based on two integration approaches. Explanation of the two approaches are covered in the Collaborative Filtering section.
- The top level recommendation function accepts an additional parameter for user-based and item-based CF tasks.
- Visualizes the results of the recommender system to compare the performances with/without Pattern Mining

# 5 Testing and Results

To evaluate the recommender system performance, we created the evaluate function, which creates a dataframe with the precision, recall, f1 score and MAP score for each user. The results were obtained by iterating over each user in the test set. For each user, the user_id and the items they bought are fetched, For new users, the cold start recommendations were provided. For every other user, the recommendation function is called and the recommendations are stored as a list. The recommendations are compared against the items they bought to determine the precision, recall, F1 score and MAP score.

The following two images demonstrate the performance of Recommender system with/without pattern mining for Item-based CF and User-based CF. There are two key details from the graphs:

1. The Hybrid approach is almost similar in performance or only slightly better than the Item based CF.
2. The user based CF outperforms the item based CF in terms of performance metrics.

The final scores are organized in a tabular format below:

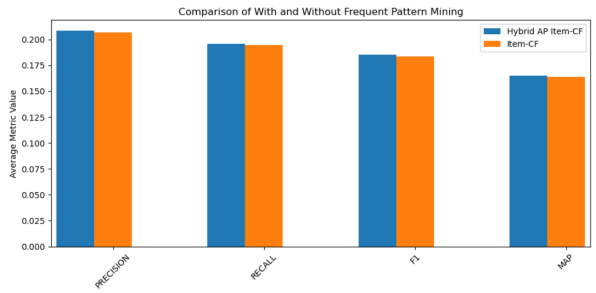



Figure 11: Performance Metrics with/without Pattern Mining (Item Based)

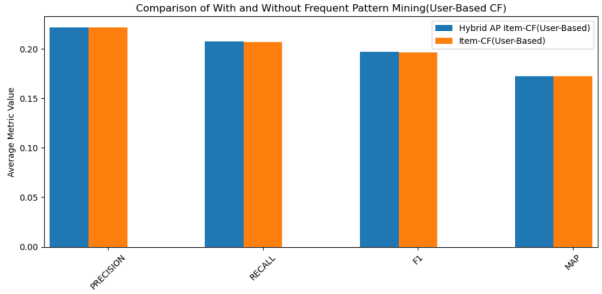



Figure 12: Performance Metrics with/without Pattern Mining (User Based)

Table 3: Top frequent patterns discovered in training and test sets

| Pattern | Support(Train) | Support(Test) | Confidence | Lift |
|---|---|---|---|---|
| (whole milk, other vegetables) | 0.0418 | 0.0387 | 0.3971 | 3.72 |
| (rolls/buns, whole milk) | 0.0356 | 0.0321 | 0.3482 | 2.67 |
| (yogurt, whole milk) | 0.0293 | 0.0275 | 0.3728 | 2.86 |
| (soda, whole milk) | 0.0273 | 0.0251 | 0.2889 | 2.21 |
| (other vegetables, root vegetables) | 0.0254 | 0.0233 | 0.3815 | 6.32 |

Table 3 outlines the frequent patterns having consistent support across training and test sets which shows the patterns are stable.

Table 4 highlights the top recommended patterns.

Table 4: Recommendations from top patterns

| Pattern | Recommended Items |
|---|---|
| (whole milk, other vegetables) | tropical fruit, yogurt |
| (rolls/buns, whole milk) | soda, butter |
| (yogurt, whole milk) | other vegetables, fruit/vegetable juice |
| (soda, whole milk) | rolls/buns, bottled water |
| (other vegetables, root vegetables) | whole milk, onions |

The performance metrics table shows the metrics(performance and timing metrics) of recommendations with/without frequent patterns.

Table 5: Performance metrics comparison

| Metric | Without Patterns | With Patterns |
|---|---|---|
| Precision | 0.2067 | 0.2084 |
| Recall | 0.1944 | 0.1959 |
| F1 Score | 0.1836 | 0.1851 |
| MAP Score | 0.1639 | 0.1648 |
| Average Time per user(ms) | 10.2 | 51.8 |

The timing analysis shown in Table 5 shows that there is a significant overhead(5 times) when using pattern mining with the recommendation system. This can cause significant issues when scaling to millions of users. The code therefore needs to be refactored to reduce the computational time for pattern mining task and integration.

# 6 Conclusions and Recommendations

## 6.1 Key Findings

The usage of both pattern mining and collaborative filtering for recommendation generation yielded some key findings. They are as follows:

1. **Recommendation System Performance**: One of the key aims of the project is to evaluate whether the recommendation system performs better with Pattern mining. Our results have shown that it is negligible, and the pattern mining integration needs to be improved to use rules when recommending items to a user. If this is rectified, we can suggest the usage of a hybrid approach when recommending to users.
2. **Pattern Mining Insights**: By tuning the support value, we have found that support values between 0.001 and 0.005 yielded better patterns. Also, the time window analysis indicated that the recent data, which is 30-90 days old, produces a better pattern.
3. **Recency Effects**: The usage of recency based scoring by the recommendation system yielded the result based on the temporal purchasing behavior of the user.
4. **Integration Benefits**: The hybrid approach outperformed collaborative filtering only models across all the evaluation metrics (precision, recall, F1 score, and MAP score). The hybrid scoring model can be improved using pattern mining rule based recommendations.

## 6.2 Method Recommendation

From our combined evaluation of all the different tasks, the following methods should be integrated into the system:

- **Pattern Mining**: Choosing support values between 0.001 and 0.005 and confidence values between 0.07-0.1 for mining using FP-growth.
- **Scoring Methodology**: The combined scoring should use the pattern rules obtained from the Pattern Mining task which should be evaluated and combined based on equal weightage scoring with CF.
- **Recency Weighting**: Use recency weighting in both task 1 and task 2. Task 1 and task 2 should use exponential decay recency weighting.
- **Time Window**: Focusing on transactions from the past 50 days for pattern mining

These methods are vital to developing a Recommender system.

## 6.3 Limitations

The key limitations are:

1. **Pattern Mining Analysis**: Since there are very few transactions in the dataset for a specific user, user level pattern mining is not effective, and we have to stick with global patterns.
2. **Cold Start Problem**: The cold start problem is addressed by providing the most popular items, but the most popular items are not referenced by geographical location or contextual factors discussed in the next point.
3. **Limited Context**: The final output does not incorporate other input features day of the week, public holidays, or day of the month, that might influence a user's shopping purpose and hence, recommendations.
4. **Performance Trade-off**: The hybrid approach increases computational complexity by a large margin, specifically 5 times more. This performance impact is non trivial considering large scale recommendation systems.

## 6.4 Significance

The developed recommendation system has significant implications for grocery retailers:

1. **Better Customer Experience**: The better a recommendation system, the happier the customers and the business due to profits.
2. **Inventory Optimization**: The pattern mining section can help us understand the association between various products, which informs inventory planning, especially during different time periods of the year.
3. **Targeted Marketing**: The combination can provide relevant recommendations which are user specific, especially with CF, even though global patterns are only available for analysis.

A small improvement in performance can create drastic revenue when scale comes into play. The code and the system could be developed further to be integrated into an actual online retail system.

# 7 Reflections

Every team member in our group contributed equally to the project report, there was proper communication, and all the team members helped each other in every aspect to complete the project.

## 7.1 (Shashi)

I made my assignment step by step, and I made several drafts before reaching the final product. Before starting I manually inspected the training data on Excel and found that most of the rows, in the end, had no values, I deleted those rows over there and proceeded with my assignment just to ease things but in a later **version 1** used the original file and have done the preprocessing by using pandas. In **version 1** of my assignment, I checked if the data was loading properly, and again I checked for missing values in order to find if my deletion was successful. Later, I just explored statistics such as unique users and items.

In **version 2**, I started building the function for preparing transactions so that it can be used in mlxted's FP-growth function. Since the assignment specifies a pattern based on user ID, I have given it functionality to perform transactional encoding on the user, too. After this, I created the mine_patterns pattern function, which uses this prepare_transactions function internally and implements the FP growth algorithm using the mlxtend package as discussed with my teammates earlier. Later, I ran some tests to check if both functions were working properly.

In **version 3**, I tried taking more insights, such as transactional count per month, and monthly transaction trends of popular items. In **version 4**, I gained more insights by finding out frequent patterns and association rules on different support and confidence levels. I found that these values are very sensitive and need to be kept very low for generating frequent itemsets and association rules. As with the discussion, I implemented recency weights calculation, which gives weights to the items that were recently purchased. It is based on time; the weight decays if the purchased item is older than the most recent purchase.

In **version 5**, I performed more exploratory data analysis, such as finding the number of users by transaction counts and basket size counts, etc. I realized that I have to make a scoring metric that not just scores based on the recency but is tunable and uses both recency and support to score the pattern, I realized that the users by transaction count are less and recency may not work properly, so I planned to introduce another method of scoring too in the score pattern function.

In **versions 6 and 7**, I started making the score_patterns pattern function in **version 6**, the scoring was done on support or confidence if the rules were provided, and in **version 7** I extended that code to do scoring based on recency or combined scoring based on support and recency.

In the final **version 8**, I ran many tests to check if everything was working correctly. I made several graphs that are useful in understanding the output of the pattern miner, and they will also help Task 2 to tune the parameters of Task 1 for a better recommendation. I also restructured the code properly, added my analysis for the codes and outputs, and created a function for Task-2, which can do both mining and scoring and can be used in integrating with a collaborative filter. This reflection, along with all my draft submissions, shows that I have learned and improved step by step while doing this assignment.

## 7.2 (Joel Varghese Thomas)

In the **first draft** of the report, I created a baseline model for collaborative filtering process which involves the creation of utility matrix, cosine similarity matrix and dot products of relevant fields for computing the final recommendations. We did not perform any evaluation in this section, as we had to add more features pertinent to the assignment requirements - Recency weighting as well as comparison between User based and Item based CF approaches.

In the **second draft** of the report, I added recency weighting to create a weighted matrix, which i then used for calculating the dot products for the recommendations. In the second draft, I also added the code to experiment with an adjusted cosine similarity function outlined in one of the reference papers. The adjusted cosine similarity produced outputs after eliminating user bias. This did not produce a good outcome as seen in the third draft.

The **third draft** of the report completed the testing and comparison between user and item-based CF methods. The evaluation metrics were selected and the user-based recommendation faired better than item based recommendations but were quite unstable. Furthermore, the adjusted cosine similarity did not provide good results and was hence, removed. The visualization of the final analysis is also provided here.

The **final version** of the report only has comments and the exploratory data analysis is added here as well for CF relevant details.

The CF section was pretty smooth sailing but resulted in some setbacks when trying out non-standard methods. Hence, to avoid complexity and poor outcomes, we stuck with the standard route without too much experimenting(we did experiment with Adjusted Cosine, recency weighted, item vs user based) around for optimization.

## 7.3 (Anoop Ajith)

The **draft one** of the report consist of mostly commented code and function definitions. The code from the collaborative filtering section is added. A simple text based UI which takes user input and an option to include or not include patterns is provided.

The **draft two** of the report combines Task 1 and Task 2 to provide the final integration code. There is also some added visualizations at the beginning of the notebook. This code only provides the top 5 recommendations based on the combination used. The technique used to integrate the pattern mining into CF was not effective and made bit better in the final integration code.

The **final** performs all the testing and evaluation of the recommendations. The integration uses the second approach mentioned in the integration section where the top 30 CF items are combined with APM items using mostly intersection to get the best items. This produced a little better output. The final code is also divided into sections, relevant sections commented and visualized the final results for metrics relevant from the CF task 2.

# References


[1] D. R. Liu, C. H. Lai, and W. J. Lee, "A hybrid of sequential rules and collaborative filtering for product recommendation," *Information Sciences*, vol. 179, no. 20, pp. 3505–3519, 2009. doi: 10.1016/j.ins.2009.06.028.

[2] K. Choi, D. Yoo, G. Kim, and Y. Suh, "A hybrid online-product recommendation system: Combining implicit rating-based collaborative filtering and sequential pattern analysis," *Electronic Commerce Research and Applications*, vol. 11, no. 4, pp. 309–317, 2012. doi: 10.1016/j.elerap.2012.02.004.

[3] I. Viktoratos, A. Tsadiras, and N. Bassiliades, "Combining community-based knowledge with association rule mining to alleviate the cold start problem in context-aware recommender systems," *Expert Systems with Applications*, vol. 101, pp. 48–61, 2018. doi: 10.1016/j.eswa.2018.01.044.

[4] Customers.ai (2025). Recency-Weighted Scoring. https://customers.ai/recency-weighted-scoring

[5] HackerTraining (2025). Association Rules: Apriori and FP-Growth. https://hackertraining.org/ai_research/ML_Fundamentals/ai_generated/data/Association_Rules_%28Apriori%2C_FP-Growth%29/

[6] Aditya (2023). Implement FP Growth Algorithm in Python. https://codinginfinite.com/implement-fp-growth-algorithm-in-python/

[7] Parvatikar, S., & Joshi, B. (2015). Online book recommendation system by using collaborative filtering and association mining. *Proc. 2015 IEEE Int'l Conf. Computational Intelligence and Computing Research (ICCIC), Madurai, India*, 1–4.